Learning the Rank of an Elliptic Curve

Lecture notes written while building `elliptic-curve-intelligence-lab`

Dylan Costa

Contents

0 The problem: which rank are we predicting? **1**

Solutions to exercises **4**

0 The problem: which rank are we predicting?

We want a function f that takes an elliptic curve $E/\mathbb{Q}$, given by its Weierstrass coefficients, and returns the probability that its rank is positive (Task 1) or, among curves of even rank, that its rank is 2 rather than 0 (Task 2). Computing the rank is a hard problem with no known algorithm guaranteed to terminate. Predicting it is a statistical question: given a few million curves whose ranks are known, which cheaply computable features of a curve carry information about its rank, and does that information survive on curves larger than the ones we trained on? This section pins down what "rank" means, which rank our labels are, and why Task 2, not Task 1, is the honest version of the question.

Curves and their rational points. An elliptic curve over $\mathbb{Q}$ is a smooth projective curve given by a Weierstrass equation

$$E:\ y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6, \qquad a_i \in \mathbb{Z},$$

with nonzero discriminant Δ. Its rational points form an abelian group, and Mordell's theorem (1922) says this group is finitely generated:

$$E(\mathbb{Q}) \ \cong\ E(\mathbb{Q})_{\text{tors}} \oplus \mathbb{Z}^r, \qquad r = \operatorname{rk} E(\mathbb{Q}).$$

The torsion part is well understood: by Mazur (1977) it is one of exactly 15 groups, and it is easy to compute. The *algebraic rank* r is the mystery. A curve has rank 0 when it has only finitely many rational points, and positive rank when it has infinitely many. Our dataset stores each curve by its label and coefficients, for example `11.a1|[0,-1,1,-7820,-263580]|0|[]|11|1`: label, $[a_1, a_2, a_3, a_4, a_6]$, rank, torsion structure, conductor, and the analytic order of the Shafarevich–Tate group Sha.

Counting points mod p, and the L-function. Reducing the equation mod a prime p gives a curve over $\mathbb{F}_p$ with some number of points. Write

$$a_p \ =\ p + 1 - \#E(\mathbb{F}_p), \qquad |a_p| \le 2\sqrt{p} \quad \text{(Hasse, 1933) for } p \nmid N.$$

Here N is the *conductor*, a product of the primes where E has bad reduction (with exponents that record how bad). The a_p assemble into the L-function

$$L(E,s) = \prod_{p\nmid N} (1 - a_p p^{-s} + p^{1-2s})^{-1} \prod_{p\mid N} (1 - a_p p^{-s})^{-1},$$

which converges for $\operatorname{Re} s > 3/2$. By the modularity theorem (Wiles; Taylor–Wiles; Breuil–Conrad–Diamond–Taylor, 2001), $L(E,s)$ extends to an entire function, and $\Lambda(E,s) = N^{s/2}(2\pi)^{-s}\Gamma(s)L(E,s)$ satisfies the functional equation

$$\Lambda(E,s) \;=\; w\,\Lambda(E,2-s), \qquad w \in \{\pm 1\}. \tag{1}$$

The sign w is the *root number*. The *analytic rank* is $r_{\mathrm{an}} = \operatorname{ord}_{s=1} L(E,s)$, the order of vanishing at the center of symmetry.

Conjecture 0.1 (Birch and Swinnerton-Dyer)**.** *$r_{\mathrm{an}} = r$. Moreover the leading Taylor coefficient is*

$$\frac{L^{(r)}(E,1)}{r!} \;=\; \frac{\Omega_E \cdot \mathrm{Reg}(E) \cdot \prod_{p\mid N} c_p \cdot \#\,\mathrm{Sha}(E)}{\#E(\mathbb{Q})^2_{\mathrm{tors}}},$$

with Ω_E the real period, Reg *the regulator of a basis of $E(\mathbb{Q})/$torsion, and c_p the Tamagawa numbers.*

The conjecture grew out of numerical experiments in the 1960s with exactly the quantity we will use: Birch and Swinnerton-Dyer observed that $\prod_{p\le X} \#E(\mathbb{F}_p)/p$ seems to grow like $C\,(\log X)^r$. Curves of higher rank have, on average, *more* points mod p, so their a_p lean negative. That is the signal a model can hope to find.

Which rank are our labels? For small ranks, the two ranks are known to agree, and this is what makes the labels trustworthy.

Theorem 0.2 (Gross–Zagier 1986, Kolyvagin 1989, plus modularity)**.** *If $r_{\mathrm{an}} \le 1$, then $r = r_{\mathrm{an}}$ and* $\mathrm{Sha}(E)$ *is finite.*

Proposition 0.3. *Suppose $r \le 3$ is proven for E (by exhibiting r independent points and an upper bound from descent), that $L^{(r)}(E,1) \neq 0$ has been certified numerically, and that the root number satisfies $w = (-1)^r$. Then $r_{\mathrm{an}} = r$.*

Proof. Since $L^{(r)}(E,1) \neq 0$, $r_{\mathrm{an}} \le r$. If $r \le 1$, then $r_{\mathrm{an}} \le 1$ and the theorem gives $r_{\mathrm{an}} = r$. If $r \ge 2$, then $r_{\mathrm{an}} \le 1$ is impossible, because the theorem would force $r = r_{\mathrm{an}} \le 1$. So $2 \le r_{\mathrm{an}} \le r$. For $r = 2$ we are done. For $r = 3$, Exercise 0.1 gives $w = (-1)^{r_{\mathrm{an}}}$, so r_{an} is odd, hence 3. □

For rank 4 the argument breaks down: $r_{\mathrm{an}} \in \{2, 4\}$, and nothing known rules out 2. That is one reason Task 2 stops at rank 2. Section 1 checks how the dataset's ranks were obtained, since the curves above conductor 500,000 come from a different source than the complete tables below it.

The parity shortcut, and why Task 1 is not enough. The root number is not a mysterious analytic quantity. It factors into local signs, $w = -\prod_{p\mid N} w_p$, and each w_p is determined by the reduction of E at p. For multiplicative reduction, $w_p = -a_p$, where $a_p = +1$ (split) or -1 (non-split). Additive reduction is more involved but still a finite local computation. So w is a cheap, deterministic function of the coefficients, and by Exercise 0.1,

$$w = (-1)^{r_{\mathrm{an}}}.$$

Every curve with $w = -1$ has odd, hence positive, rank. A flexible model trained on Task 1 can therefore get a large part of the way by rediscovering the parity of the rank from the bad primes, which is a theorem rather than a discovery. Dropping a "root number" column does not fix this, because the model can rebuild w from features we do allow. The fix is to change the question so that parity carries no information:

task	curves	label
Task 1	all	$r > 0$
Task 2	$w = +1$, i.e. $r \in \{0, 2\}$	$r = 2$

In Task 2, w is constant, so any skill a model shows must come from somewhere else, and the Birch–Swinnerton-Dyer heuristic says where to look: the a_p. Because w is known and cheap, filtering the curves in the code is a simple rank filter:

From src/eclab/tasks.py

```
TASK2_RANKS = (0, 2)

def task2_mask(rank) -> np.ndarray:
    """Which curves belong to Task 2: those of rank 0 or 2."""
    return np.isin(np.asarray(rank), TASK2_RANKS)
```

We keep Task 1 anyway, for two reasons: it is the question most people would ask first, and comparing a model's score on Task 1 with its score on Task 2 measures how much of its Task 1 skill was just parity.

What the model may not see. Several quantities are computed *from* the rank, or from the L-function at $s = 1$, and would turn the experiment into a lookup. Section 2 audits every feature; the main exclusions are already clear from the BSD formula:

- $L(E, 1)$ and the leading coefficient: $L(E, 1) \neq 0$ if and only if $r_{\text{an}} = 0$.
- The regulator and generators of $E(\mathbb{Q})$: they exist only once the rank is known.
- The analytic order of Sha (the last column of our data): it is computed from the BSD formula, which needs the rank and the leading coefficient.

Torsion, the conductor, the discriminant, Tamagawa numbers and the a_p are allowed. They are properties of the curve or its reduction mod p, not of the rank.

What success means here. A high score is not the goal. Ranks are already known for every curve in the dataset, so the point is the scientific question: how much of the rank is visible in the a_p for small p, and does what a model learns at small conductor carry over to large conductor? A careful "not much, and here is why" is a result. This question is not new: He, Lee, Oliver and Pozdnyakov trained classifiers on a_p data from the LMFDB and found, by looking at what the classifiers used, a new phenomenon in the averages of a_p by rank ("murmurations", 2022). Section 7 places our results relative to theirs. What these notes add is the parity-free Task 2, the out-of-distribution test, and the leakage audit.

Roadmap. 1 data and validation · 2 features and the leakage audit · 3 grouped and out-of-distribution splits · 4 baselines · 5 gradient boosting and ablation · 6 a small neural network and calibration · 7 larger conductors and murmurations · 8 elliptic-curve signatures in practice · 9 post-quantum signatures · 10 reproducibility · 11 talking about it.

How the repo maps to these notes. The package is `src/eclab/`, tests are in `tests/`, and every number and figure from Section 1 on is produced by `scripts/figs_stepNN.py` and pulled in with `\input`, so the notes cannot silently disagree with the code.

Exercises.

Exercise 0.1. Expand $\Lambda(E,s)$ in a Taylor series around $s=1$ and use (1) to show that $w=(-1)^{r_{\mathrm{an}}}$. (Check first that the factor $N^{s/2}(2\pi)^{-s}\Gamma(s)$ does not vanish at $s=1$.)

Exercise 0.2. Suppose E is semistable, so every bad prime is multiplicative and N is squarefree. Show that $w=(-1)^{\omega(N)+1}\prod_{p\mid N}a_p$, where $\omega(N)$ is the number of primes dividing N. Then check the formula on `11.a` (rank 0, split at 11) and `37.a` (rank 1, non-split at 37).

Exercise 0.3. In Task 2 let π be the fraction of rank-2 curves. Show that the best constant predictor has accuracy $\max(\pi,1-\pi)$ and log-loss $H(\pi)=-\pi\log\pi-(1-\pi)\log(1-\pi)$. Explain why accuracy alone makes a useless model look good when π is small, and why we report log-loss and PR-AUC as well.

Interview prep. *In 60 seconds:* "The rank of an elliptic curve is the number of independent infinite-order rational points. The obvious ML task, zero versus positive rank, is partly solved by parity: the root number is a cheap local computation and equals (-1) to the rank. So I made the headline task rank 0 versus rank 2 among curves with root number $+1$, where parity carries no signal, and tested whether what the model learns transfers to larger conductors." *A skeptic asks:* "Aren't the ranks just conjectural?" Answer with Proposition 0.3: for ranks up to 3 with proven algebraic rank, analytic and algebraic rank provably agree; that is why the tasks stop at rank 2 and leave rank 4 out.

Solutions to exercises

Exercise 0.1. The gamma factor is nonzero at $s=1$ ($\Gamma(1)=1$), so $\operatorname{ord}_{s=1}\Lambda=r_{\mathrm{an}}$ and $\Lambda(E,s)=c\,(s-1)^r+O((s-1)^{r+1})$ with $c\neq 0$ and $r=r_{\mathrm{an}}$. Substituting $s\mapsto 2-s$ gives $\Lambda(E,2-s)=c\,(1-s)^r+\cdots=(-1)^r c\,(s-1)^r+\dots$. Comparing leading coefficients in (1) gives $c=w(-1)^r c$, so $w=(-1)^r$.

Exercise 0.2. With $w=-\prod_{p\mid N}w_p$ and $w_p=-a_p$ for every $p\mid N$, $w=-\prod_{p\mid N}(-a_p)=-(-1)^{\omega(N)}\prod a_p=(-1)^{\omega(N)+1}\prod a_p$. For `11.a`: $\omega=1$, $a_{11}=1$, so $w=+1$, consistent with rank 0. For `37.a`: $\omega=1$, $a_{37}=-1$, so $w=-1$, consistent with rank 1.

Exercise 0.3. A constant prediction of class 1 is right on a fraction π of curves and class 0 on $1-\pi$; take the larger. A constant probability q has expected log-loss $-\pi\log q-(1-\pi)\log(1-q)$, minimized at $q=\pi$ with value $H(\pi)$. When π is small, predicting "rank 0" always scores $1-\pi$, close to 1, while finding no rank-2 curve at all. Log-loss rewards confidence that is justified, and PR-AUC measures how well the model ranks the rare class, so both expose that predictor.